

Coug Assets

KADEN METZGER & TRAVIS TAKUSHI

MAY 1ST, 2026

Coug Asset Objectives

- Create a working web application that EECS staff and admin can use
- Streamline the asset sharing process within the EECS department
- Create an efficient and easy to use system

Problem Statement & Scope

- The EECS department at WSU has assets that are shared within their department. Often times, sharing these assets can be difficult. The current system in place is outdated and inefficient, causing some assets to be purchased that are already present. Our project aims to solve this problem.
- Our target audience is anyone within the WSU EECS department whether they are teachers or students.

Technologies Used

- Coug Assets is being developed with Next.js and PostgreSQL for the our database
- Next.js is a react based framework
- Next.js and react use something known as hooks and states for UI rendering
- Next.js aims to collapse the full stack dev experience into a singular layer for simpler, more efficient development

Functional Requirements

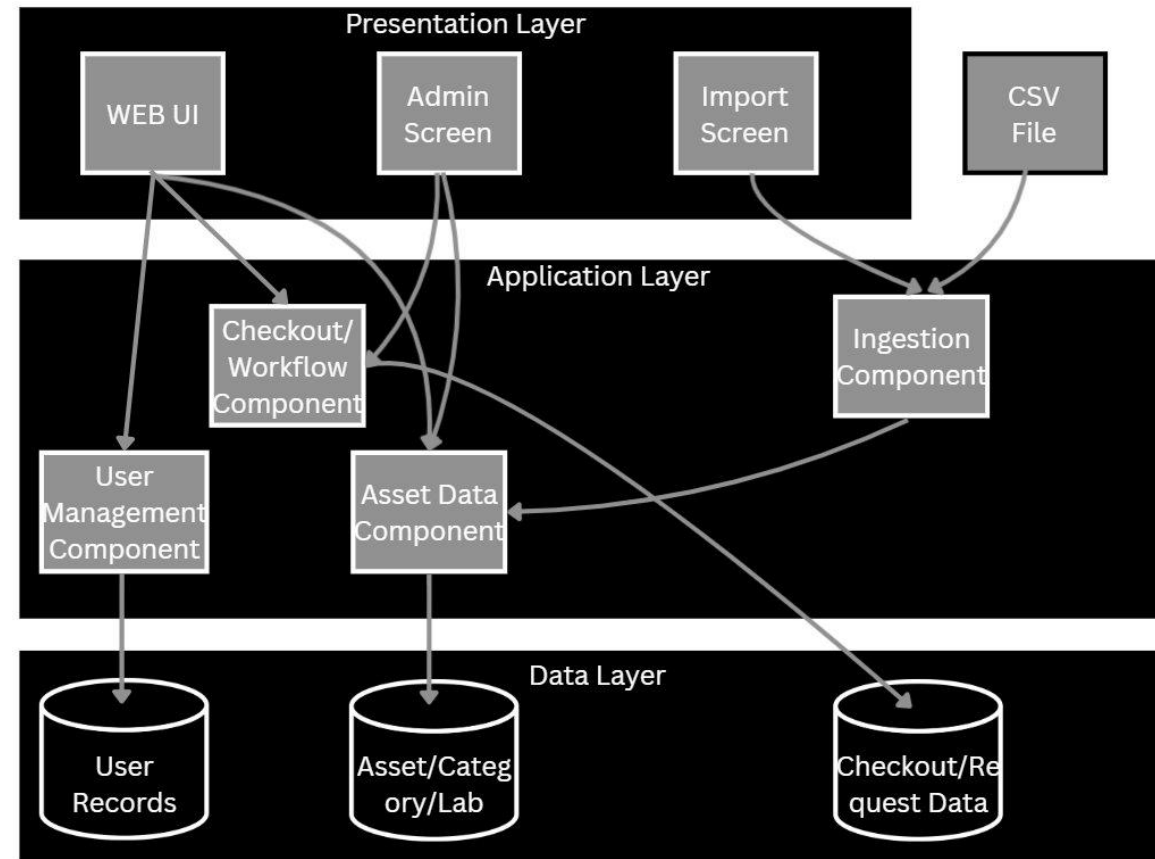
- User Login
- Asset Search
- Asset Checkout
- Admin Dashboard
- Interfacing with Database
- Routing
- Security

Non-Functional Requirements

- Usability
- Security
- Performance
- Documentation

System Architecture

- UML Diagram



System Architecture

- Our architecture features 3 layers, UI or presentation layer, application layer, and the database layer
- Our main flow:
 - User interacts with the UI layer
 - UI layer directly runs the application layer software
 - Application layer communicates with the database layer
 - Database layer fetches, updates, removes, etc entries from the database depending on the function called

Sprint 3 Achievements

- Created a start to finish workflow of creating an asset, all the way to checking out and returning the asset for both admin and users
- Polished up user interface to be more functional
- Created quick querying
- Enabled editing of assets

System demonstration

- Admin dashboard
 - Approve, Deny, Mark return
 - Create/Edit assets
- Homepage
 - Search/Filter Assets
 - Create asset request
 - View asset details
 - View current checked out assets
 - View Current Checked out requests
 - For admin
 - Admin page
 - Upload page
- Login
 - Basic login functionality

Testing

- For sprint 3 we started on unit testing using Jest.js
- We conducted basic testing of pure functions such as our asset filtering function
- Conducted unit tests of our UI, things like testing if buttons work correctly, if UI displays expected results on load, etc
- Some challenges we encountered were working with Jest. Testing a web application is different than how you would test other programs so that took some time to learn

Responsible Use of AI

- Use of chatgpt and github copilot to ask questions when stuck, as well as proper formatting and optimization of systems

Plan and Deliverables for Next Semester

- Functional Requirement Plans:

- Finish admin dashboard
- Implement barcode scanning
- Polish all features and conduct tests to help identify bugs

- Non-Functional Requirement Plans:

- Redesign the Admin Approve/Deny menu to prep for scaling issues
- Polish UI
- Discuss with client about the web application's accessibility and ease of use

Next Semester Timeline and Deliverables

- Timeline:

- Sprint 4: Finish all main functionality and conduct all testing phases
- Sprint 5: Integrate with OKTA and VM, polish UI, keep refining the site
- Sprint 6: Add final touches, recap functional and non-functional requirements to ensure clients needs are met, and finally deliver a fully working product

- Final Deliverables:

- A working product that is accessible and intuitive to use
- Our site must be integrated with OKTA and usable by staff and students
- Fully fleshed out features that make the site easy to use and manage

Conclusion

- Our product is up to par with client standards so far
- Created a minimum viable product that can be tested with users